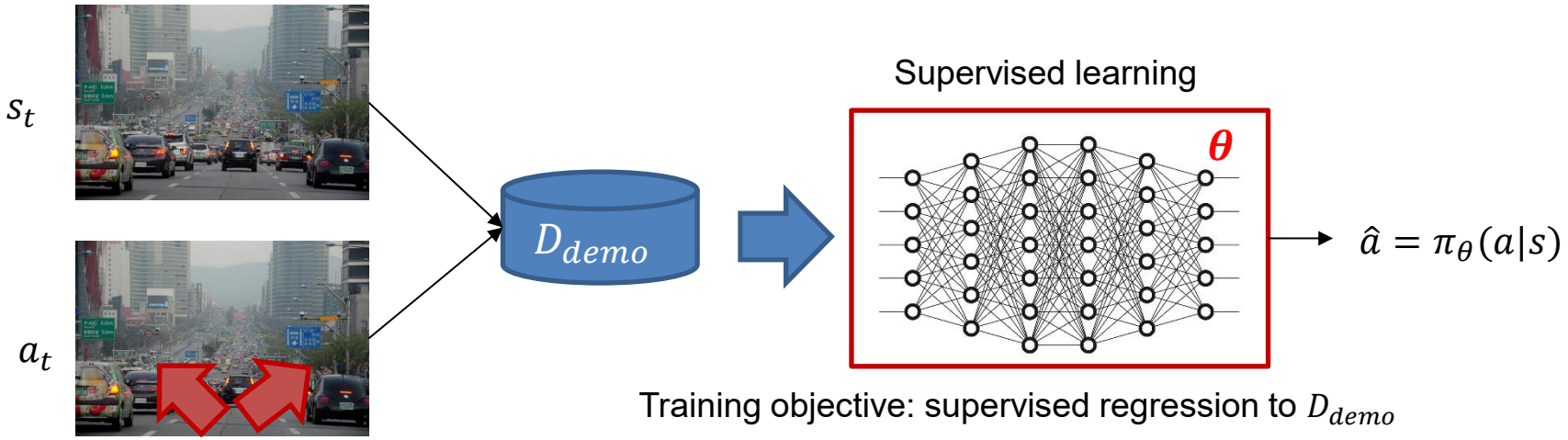
Imitation Learning

- Behavior Cloning
- Problems
  - Non i.i.d data
  - Compounding error
  - Multi-modal action distribution
- Approaches for better behavior cloning
  - Be smart about how we collect (and augment) our data
  - Use very powerful models that make very few mistakes

Learning from Demonstrations

- *Expert* provides a set of *demonstration trajectories:*
  - sequences of states and actions
    - $D_{demo} = \{(s_1, a_1), (s_2, a_2), \dots, (s_T, a_T)\} \sim \pi_{EXP} = p_{EXP}(a|s)$
  - But *no reward function*
- Imitation learning is useful *when it is easier* for the expert to demonstrate the desired behavior rather than:
  - Specifying a reward that would generate such behavior,
  - Specifying the desired policy directly



Training objective: supervised regression to D_{demo}

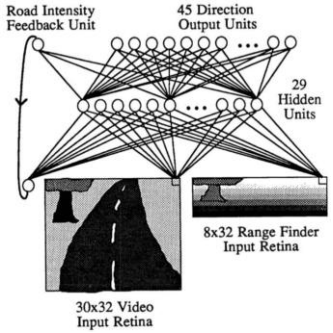
$$\min_\theta \frac{1}{|D_{demo}|} \sum_{(s,a) \in D_{demo}} \|a - \hat{a}\|^2$$

Behavior Cloning

Behavior Cloning

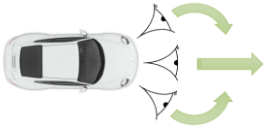
- Reduce problem to a standard supervised machine learning problem:
  - Fix a policy class (e.g. neural network, decision tree, etc.)
  - Estimate a policy from training examples $(s_0, a_0), (s_1, a_1), (s_2, a_2), \ldots$
- Examples
  - ALVINN (Autonomous Land Vehicle In a Neural Network), NIPS 1989

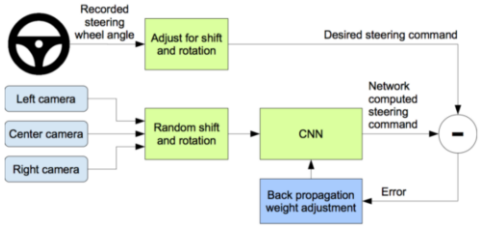






  - Learning from Demonstration for Autonomous Navigation in Complex Unstructured Terrain, Silver et al., 2010
  - Nvidia 2017
    - End-to-End Deep Learning for Self-Driving Cars
      - https://developer.nvidia.com/blog/deep-learning-self-driving-cars/
      - https://youtu.be/NJU9ULQUwng



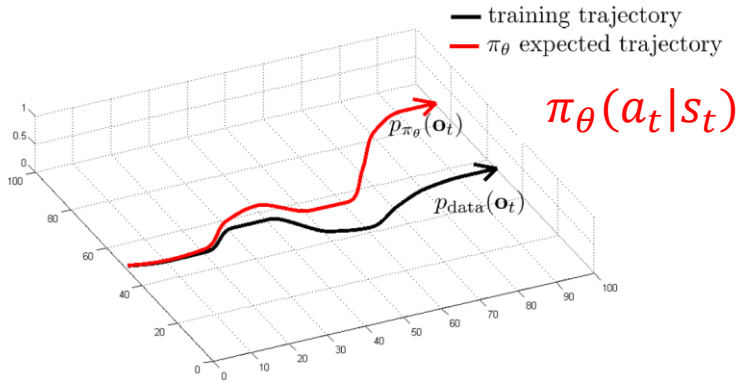


Is it end of the story?

- BC can be successful
  - Ex) End-to-End Deep Learning for Self-Driving Cars
    - https://developer.nvidia.com/blog/deep-learning-self-driving-cars/
    - https://youtu.be/NJU9ULQUwng
- However, Imitation learning via behavioral cloning is *not guaranteed* to work
  - This is *different from supervised learning*
  - In supervised learning
    - both train and test data comes from the same distribution (dataset)
  - In (offline) imitation learning (i.e., behavior cloning)
    - train data $\sim D_{demo}$ (with distribution $s_t \sim p_{demo}(s)$) while test data $\sim \pi_\theta$ (with distribution $s_t \sim p_{\pi_\theta}(s)$)
      - Distribution shift: $p_{demo}(s) \neq p_{\pi_\theta}(s)$
    - *i.i.d. assumption does not hold*
      - There are temporal structure: $s_t \rightarrow a_t = \pi_\theta(a|s_t) \rightarrow s_{t+1}$
      - Predicted actions affect the next state
      - Errors can lead to drift away from $p_{demo}(s)$ that produces D_{demo}
      - Errors can be compounded
    - *Multi-Modal action distribution* in Demo dataset (D_{demo})
      - Composed of many experts who may take different action at the same state
      - L2-regression cannot reflect true action distribution
    - Data in D_{demo} is not optimal

Distribution Shift Problem

- Distribution Shift Problem
  - We train a policy under the given data (trajectories shown by experts)
    - i.e., we tried to find *policy parameter* θ *to maximize* $E_{s_t \sim D_{demo}}[\log \pi_\theta(a_t|s_t)]$
  - However, the trajectories taken by the learned policy π_θ may not be the same as the one taken by the expert
  - $p_{D_{demo}}(s_t) \neq p_{\pi_\theta}(s_t)$

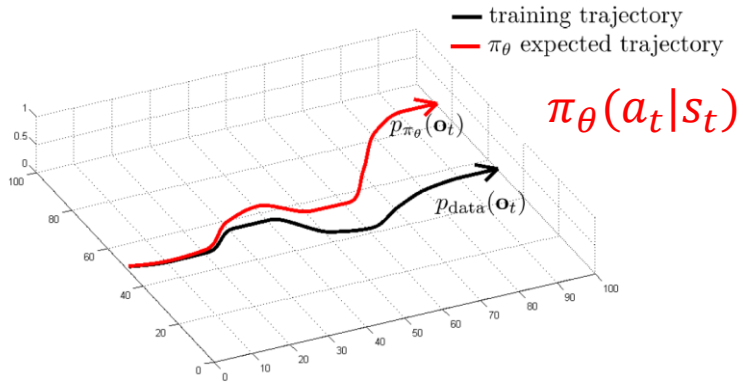


$\pi_\theta(a_t|s_t)$

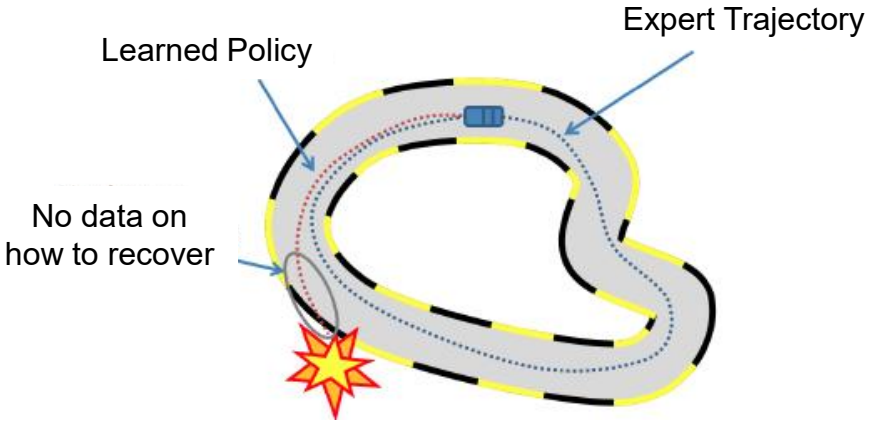
- In supervised learning,
  - Both train and test data $(x, y) \sim D$ during train and test comes from the same dataset
  - Next input is *independent* of current output
- In BC, states (s_t) comes from
  - D_{demo} when training, but
  - D_{π_θ} when testing
  - $p_{D_{demo}}(s_t) \neq p_{\pi_\theta}(s_t)$
  - Next state is *dependent* on current action

- Thus, We want to
  - *minimize* the number of *mistakes* the policy makes *when we run it*, *instead of* $\max_\theta E_{s_t \sim D_{demo}}[\log \pi_\theta(a_t|s_t)]$

❑ Example) Failure Case



$\pi_\theta(a_t|s_t)$



Compounding Error

- Error can be accumulated as time goes by
- Compounding Error (Intuitive Analysis)
  - Let's denote $\pi_{EXP}(s_t)$ be the policy given by the expert
  - define the following cost function
    - $c(s_t, a_t) = \begin{cases} 0 & if\ a_t = \pi_{EXP}(s_t) \\ 1 & otherwise \end{cases}$
  - Error at time t with probability $\leq \epsilon$
    - i.e., $\pi_\theta(a \neq \pi_{EXP}(s)|s) \leq \epsilon$



  - If independent in time errors, then
    - $E[\sum_t c(s_t, a_t)] \leq \epsilon T + (1-\epsilon)(\epsilon(T-1) + (1-\epsilon)\ldots))$
    - Since the left term repeats T times and each $O(\epsilon T)$, $E[total\ errors\ (cost)] \leq \epsilon T$
  - Thus, compounding errors
    - *$O(\epsilon T^2)$: quadratic increase over time T*

Reference
https://www.cs.cmu.edu/%7Esross1/publications/Ross-AIStats10-paper.pdf
https://www.cs.cmu.edu/%7Esross1/publications/Ross-AIStats10-sup.pdf

(Side Slide) Proof

- Assume
  - $\pi_\theta(a \neq \pi_{EXP}(s)|s) \leq \epsilon$
- For all $s_t \in D_{demo}$ (i.e., $s \sim p_{demo}(s)$)

$$p_\theta(s_t) = (1-\epsilon)^t p_{demo}(s_t) + (1-(1-\epsilon)^t) p_{mistake}(s_t)$$

Prob(no mistakes until t)

Other distribution than $p_{demo}(s)$

- Then,

$$|p_\theta(s_t) - p_{demo}(s_t)| = (1-(1-\epsilon)^t)|p_{mistake}(s_t) - p_{demo}(s_t)| \leq 2(1-(1-\epsilon)^t) \leq 2\epsilon t$$

Max($|p_{mistake}(s_t) - p_{demo}(s_t)|$)=2

Useful identity: $(1-\epsilon)^t \geq 1-\epsilon t$, for $\epsilon \in [0,1]$

- Therefore

c_{max} =Max(c_t)

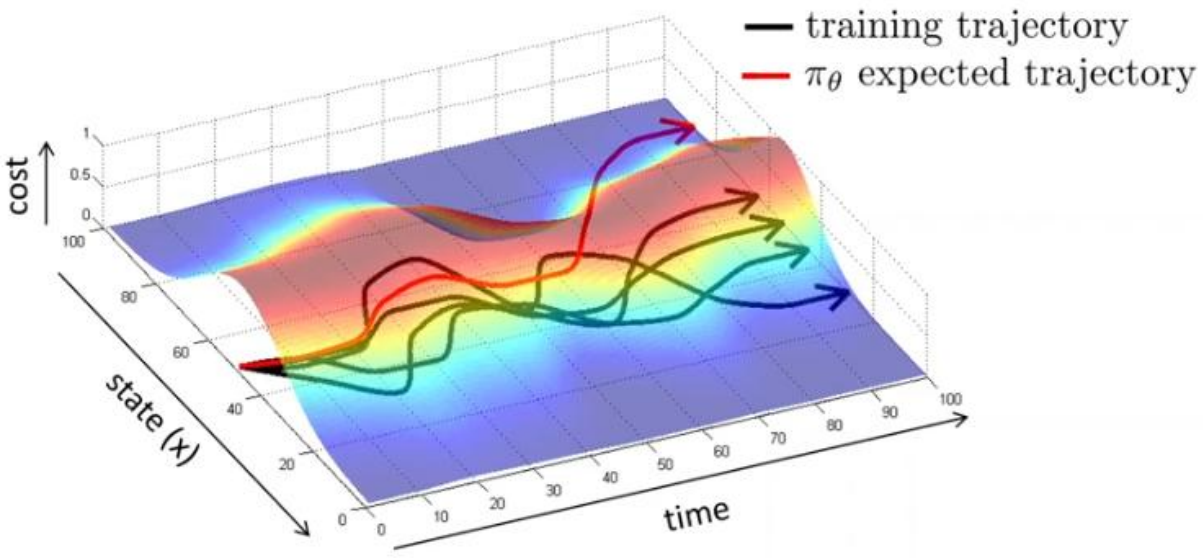
$$\sum_t E_{p_\theta(s_t)}[c_t] = \sum_t \sum_{s_t} p_\theta(s_t)\, c_t$$

$$\leq \sum_t \sum_{s_t} p_{demo}(s_t)\, c_t + |p_\theta(s_t) - p_{demo}(s_t)| c_{max}$$

$$\leq \sum_t \epsilon + 2\epsilon t \sim O(\epsilon T^2)$$

$O(\epsilon T^2)$ is rather pessimistic

- Human *can* often *recover* from mistakes
- However, that *doesn't mean* that imitation learning will *allow us to learn how to do that* !!
- Paradox
  - Imitation learning can work better if the data has more mistakes (and recoveries)

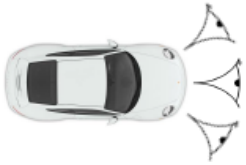


Practical Method for IL Problem

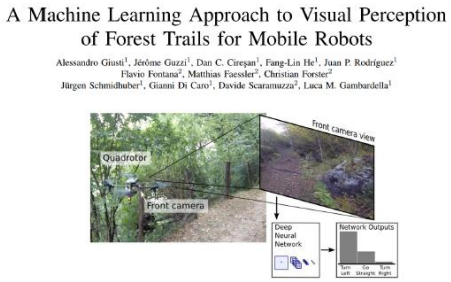
- smart data collection and data augmentation
- powerful models that consider
  - Sequential (Non-Markovian) Data Characteristics
    - Consider temporal relation among data
    - State embedding by considering observation history (Sequential data property)
  - Multi-Modal Action Distribution
    - Action generation by expressive state/action value representation

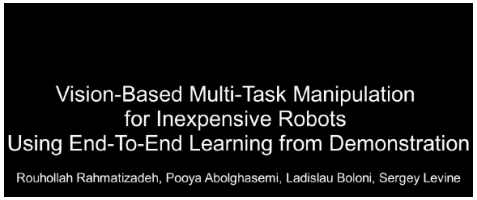
Smart Data Collection & Augmentation

- *Intentionally* add mistakes and corrections
  - Mistakes hurts
  - Corrections help often more than the hurt done by the mistakes
- Use data augmentation
  - *Add some fake data* that illustrates corrections
    - Ex. Side-face cameras



- Case Studies
  - Trail following as classification
    - https://www.youtube.com/watch?v=umRdt3zGgpU
  - Imitation with a cheap robot
    - https://www.youtube.com/watch?v=52QTL7v_-vo

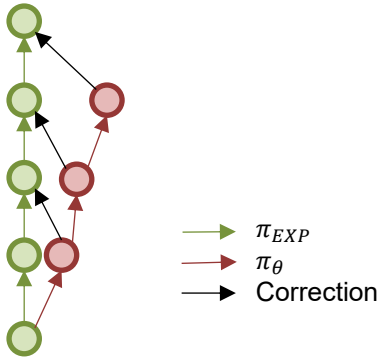




DAgger (Dataset Aggregation)

Algorithm. Collect corrective behavior data

1. Collect data by rolling out leaned policy π_θ: $s'_1, a'_1, \ldots, s'_T$
2. Query expert action at visited state $a^* \sim \pi_{EXP}(\cdot\,|s')$
3. Aggregate corrections with existing data: $D_{demo} \leftarrow D_{demo} \cup \{(s', a^*)\}$
4. *Update* policy : $\min_\theta loss(\pi_\theta, D_{demo})$

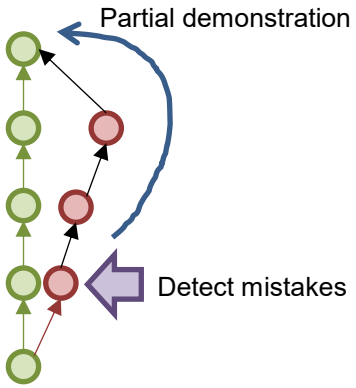


- Pros: data-efficient way
- Cons: can be challenging to query expert when agent has control

HG(Human Guided)-DAgger

Algorithm. Collect corrective behavior data with *human intervention (control)*

1. Collect data by rolling out leaned policy π_θ: $s'_1, a'_1, \dots, \boldsymbol{s'_t}$
2. *Expert intervenes* at the time (say, t) when π_θ mistakes
3. Expert provides (partial) demonstration: $\boldsymbol{s'_t}, \boldsymbol{a^*_t}, \dots, s'_T$
4. Aggregate corrections with existing data: $D_{demo} \leftarrow D_{demo} \cup \{(s'_i, a^*_i) : i \geq t\}$
5. *Update* policy : $\min_\theta loss(\pi_\theta, D_{demo})$



- Pros: more practical interface for providing corrections (ex. $\pi^*_{0.6}$, etc.)
- Cons: can be difficult to catch mistakes quickly for some tasks

(Q) Can we detect mistakes automatically?

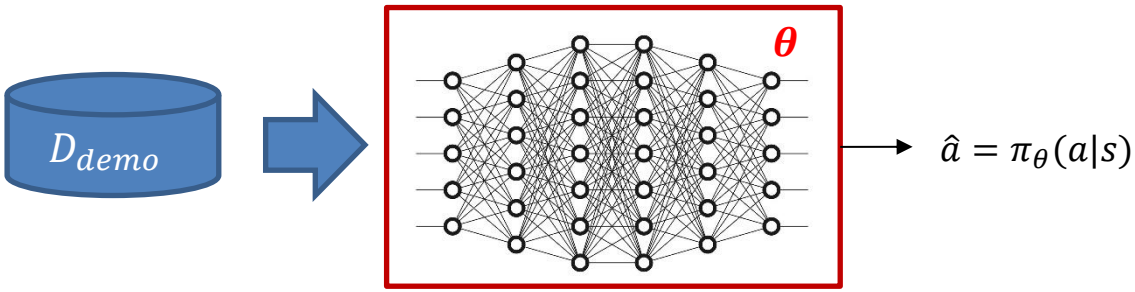
Practical Method for IL Problem

- smart data collection and data augmentation
- powerful models that consider
  - Sequential (Non-Markovian) Data Characteristics
    - Consider temporal relation among data
    - State embedding by considering observation history (Sequential data property)
  - Multi-Modal Action Distribution
    - Action generation by expressive state/action value representation

Reasons to fail to fit the expert

- Non-Markovian Behavior
  - We *train* a model based on the Markov assumption : $\pi_\theta(a_t|s_t)$
    - i.e., behavior depends only on current observation (or state)
    - If a model see the same thing twice, it do the same thing twice, regardless of what happened before
  - *Expert doesn't necessarily* choose the action based on current state
    - i.e., behavior may depend on all past observations (or states) : $\pi_\theta(a_t|s_1, \dots, s_t)$
    - Ex) when driving, human decides actions based on what he saw in the blind spot

Supervised learning



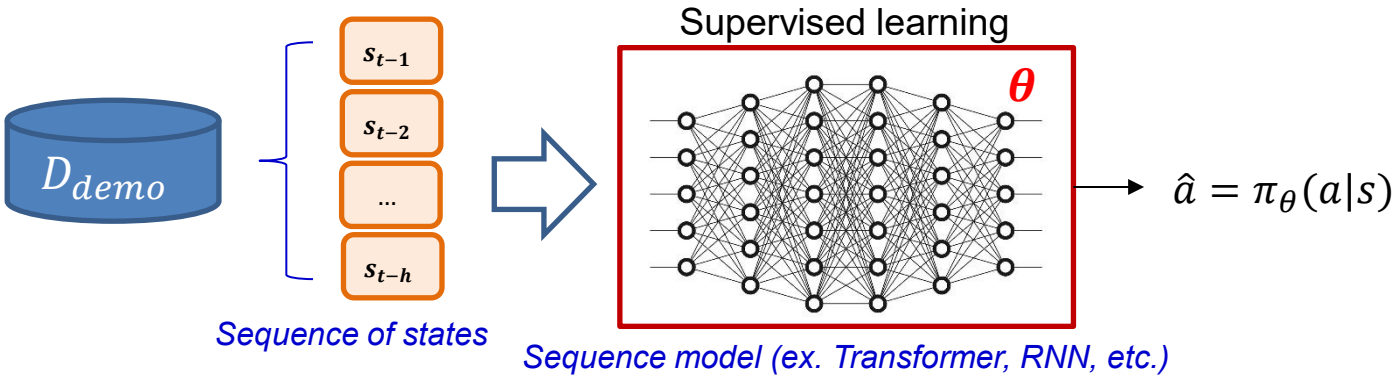
Training objective: supervised regression to D_{demo}

$$\min_\theta \frac{1}{|D_{demo}|} \sum_{(s,a)\in D_{demo}} \|a - \hat{a}\|^2$$

- Multimodal Behavior
  - Distribution of actions that an expert takes might have multiple modes

Non-Markovian Demonstration

- Use whole history by using a sequence model



- Caveat
  - Using histories of observation does not always make things better.
  - Sometimes, it makes things worse by exacerbating the correlations in the data
  - Ref) causal confusion in imitation learning,
    - https://proceedings.neurips.cc/paper_files/paper/2019/file/947018640bf36a2bb609d3557a285329-Paper.pdf

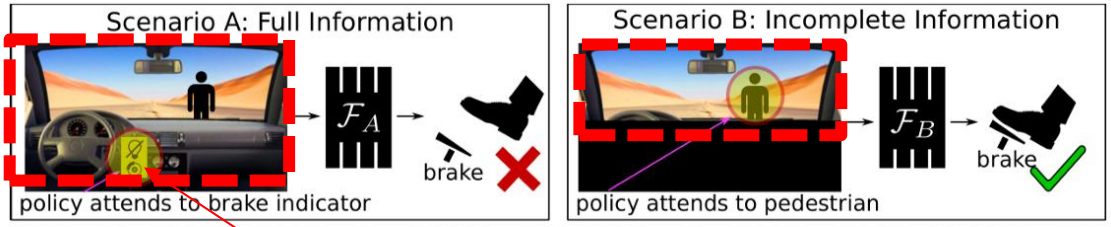


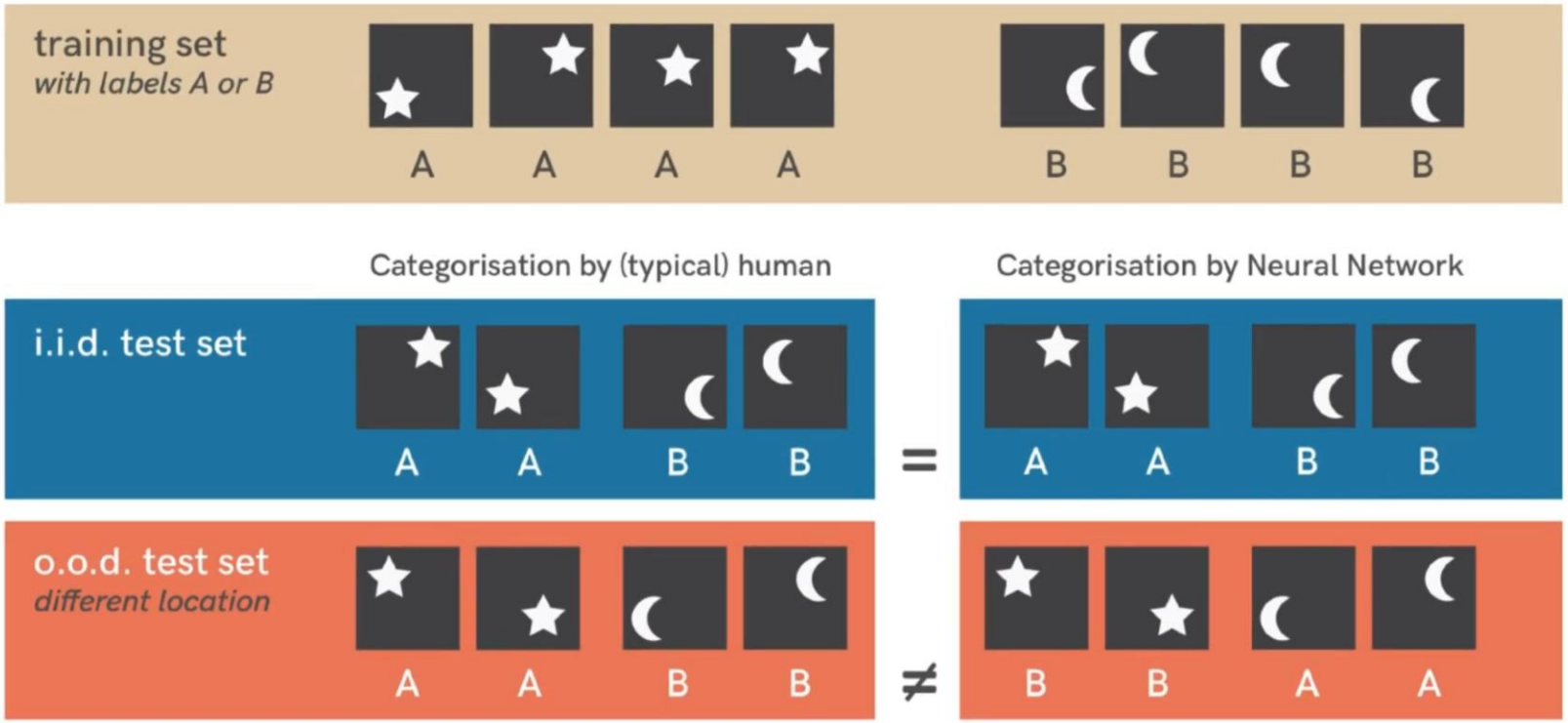
Figure 1: Causal misidentification: *more* information yields worse imitation learning performance. Model A relies on the braking indicator to decide whether to brake. Model B instead correctly attends to the pedestrian.

Input to the Model

- F_A: dash board + windshield
- F_B: windshield

Break indicator is not the cause but the effect of action (i.e., breaking) ← expert demonstration
But, model F_A misidentifies the effect as cause.

(참고) Shortcut Learning Problem

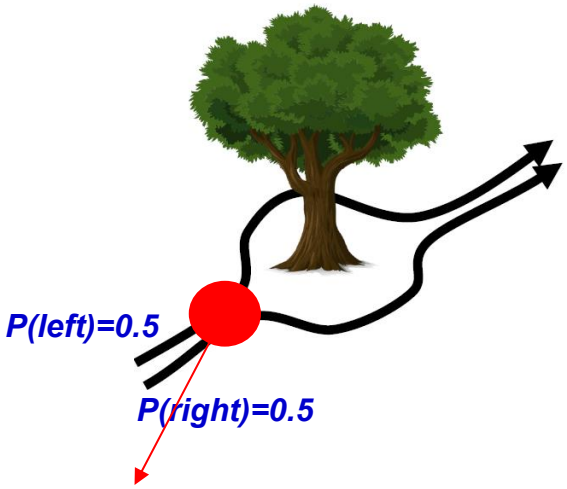


Moon	Star
Star	Moon

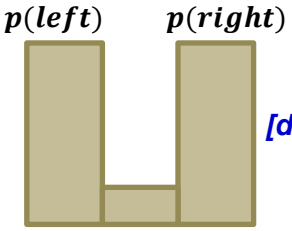
- We ***want*** a model to learn shape
  - We ***cannot*** formulate this goal
- But model learns something else (ex. Location)
- Because we are grounded in physics(objects), culture, biology

Multimodal Behavior

- Distribution of actions that an expert takes might have multiple modes
  - D_{demo} contains data from ***multiple*** experts (people, simulations, etc.)
  - Each expert may take *different* actions in the *same* state
- Ex) navigate around a tree

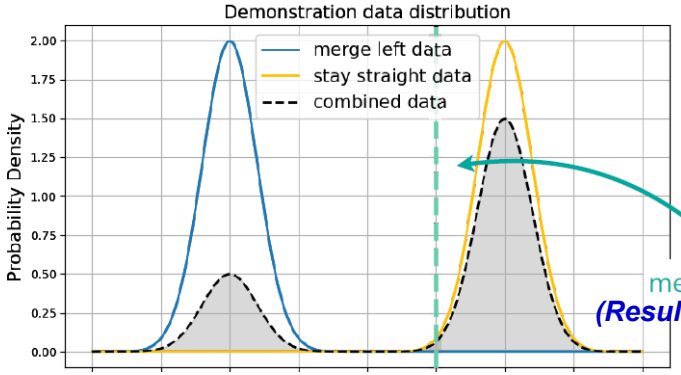


At this point, expert demonstration data includes very different actions for very similar states



[discrete action space: ok] ← deep learning model: action classification problem

[continuous action space: not ok] ← deep learning model: L2-regression problem

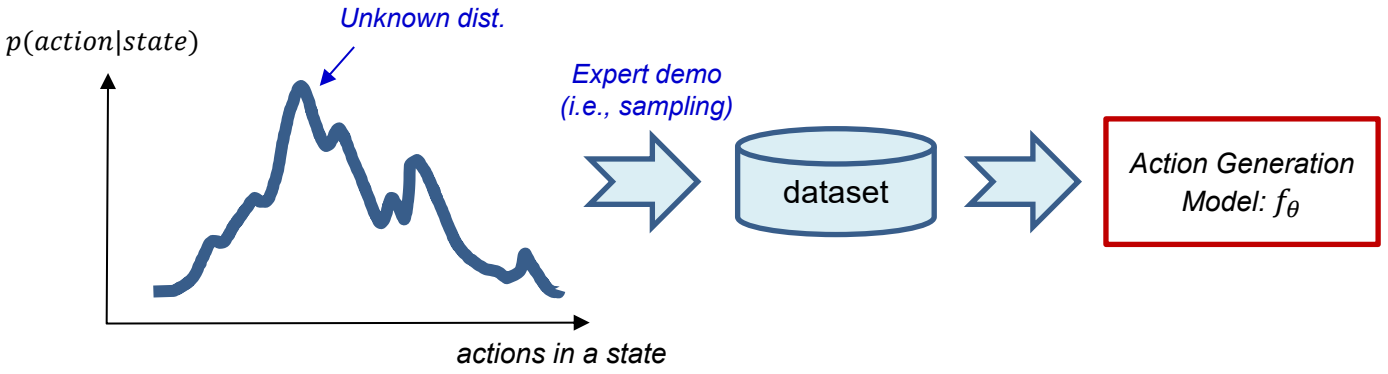


This happens in practice all the time
(Exp. when data is collected by multiple people)

We need more expressive model that can capture the ***multi-modal behavior($\pi_\theta(a|s)$)*** *in data*

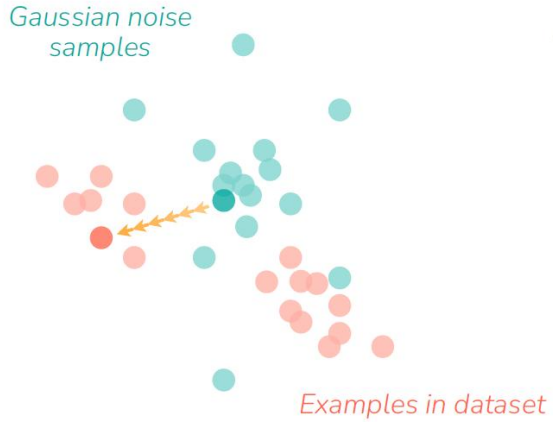
Use Expressive Action Model

- Goal
  - Train expressive policy by using generative modeling on expert demo dataset D_{demo}
- Classical Methods
  - Mixture of Gaussians Model
  - Discretize + Autoregressive
    - Discretize actions $a_i \in \{a_{i,1}, \dots, a_{i,n}\}$
    - Generate $a_i (i = 1, \dots, m)$ sequentially
      - Actions needed; $a(t) = \{a_1(t), \dots, a_m(t)\}$
      - Model
      - Generate $\hat{a}_1 \sim p(a_1(t)), \hat{a}_2 \sim p(a_2(t)|\hat{a}_1), \hat{a}_3 \sim p(a_3(t)|\hat{a}_1, \hat{a}_2), \dots$
- Recently
  - Generative Models: diffusion, flow-matching, …



Ex) Action Generation w Flow-Macthing

Images from https://cs224r.stanford.edu/slides/02_cs224r_imitation_2026.pdf



Instead: Learn to predict vector field v_θ, for iterative denoising

Training time loop:

1. Randomly sample datapoint $x_1 \sim D$ and noise $x_0 \sim \mathcal{N}(0,I)$
2. Sample $t \sim p(t)$ (e.g. $p(t) = \text{Unif}[0,1]$)
3. Linearly interpolate $x_t = tx_1 + (1-t)x_0$
4. Minimize $\|v_\theta(x_t, t) - (x_1 - x_0)\|^2$

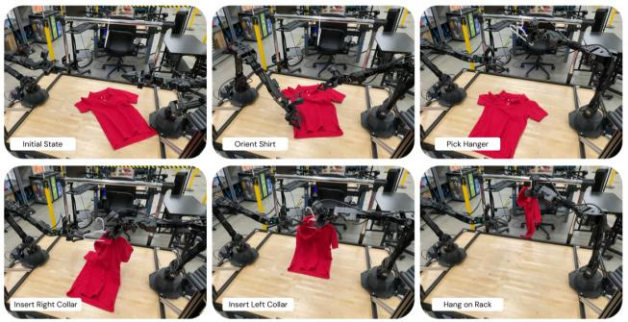
Test time

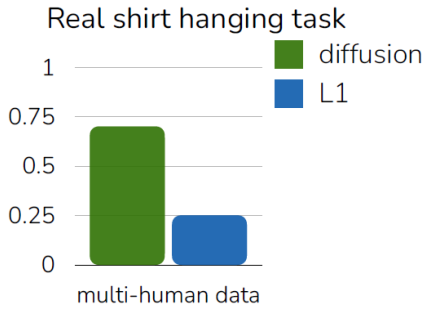
1. Randomly sample noise $x_0 \sim \mathcal{N}(0,I)$
2. For t $\in \{0, \delta, 2\delta, 3\delta, \ldots, 1-\delta\}$,
   1. $x_{t+\delta} \leftarrow x_t + v(x_t, t)\delta$ *(integrate the ODE with the Euler method)*
3. Return x_1

For imitation learning: learn conditional vector field for denoising actions a_t (conditioned on state s_t)

Example: ALOHA unleashed: A Simple Recipe for Robot Dexterity

https://aloha-unleashed.github.io/





Wrap Up

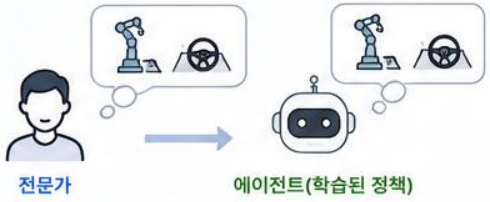
- Behavior Cloning
  - Train policy to clone offline demonstration
  - Fully offline algorithm
  - Simpler than online method
    - Online data is expensive
- Data Aggregation
  - Dagger, HG-Dagger
  - Improve policy using online intervention
  - More reliable than offline BC
- Imitation Learning
  - Pros
    - no need to define reward function
  - Cons
    - Need large amount of data for reliable performance (← maybe impractical)
    - can not improve from its own practice
      - ***Cannot outperform the policy that collected the dataset***

Imitation Learning + Reinforcement Learning

Imitation Learning (모방학습) 이란?

전문가(또는 시연자)의 행동을 관찰한 데이터를 이용해, 그와 유사한 행동을 하는 정책(에이전트)을 학습하는 방법

1 핵심 아이디어



전문가가 '어떤 행동을 하는지'를 보고 배우는 것
(보상함수를 몰라도 학습 가능)

2 학습 절차

1. 시연 데이터 수집
전문가가 환경에서 행동하는 모습을 녹화



데이터셋 $\mathcal{D} = \{(s_1, a_1), \ldots, (s_N, a_N)\}$

→

2. 정책 학습
데이터를 이용해 정책 $\pi_\theta(a|s)$ 학습



→

3. 실행(배포)
학습된 정책을 사용하여 새로운 상황에서 행동



목표: 전문가의 행동 분포(또는 상태-행동 분포)를 모방

3 수식적 정의 (Behavior Cloning 예시)

시연 데이터: $\mathcal{D} = \{(s_i, a_i)\}_{i=1}^{N}$ (상태 s_i 에서 전문가가 선택한 행동 a_i)

정책: $\pi_\theta(a|s)$ (매개변수 θ)

목표(손실 함수):

$$L(\theta) = -\frac{1}{N}\sum_{i=1}^{N} \log \pi_\theta(a_i \mid s_i)$$

최대우도(음의 로그우도 최소화)로
전문가가 한 행동의 확률을 높이도록 학습

4 대표적인 방법들

1) Behavior Cloning (BC)

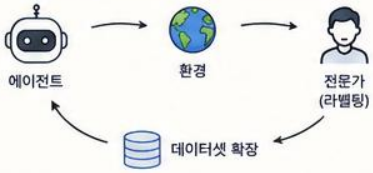
가장 기본적인 방법. 상태를 입력으로 전문가가 선택한 행동을 직접 예측



지도학습(분류/회귀)으로 학습

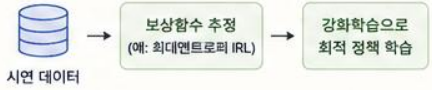
2) DAGGER (Dataset Aggregation)

오류 분포를 줄이기 위해, 학습 중 에이전트의 상태를 전문가가 라벨링하여 데이터를 확장



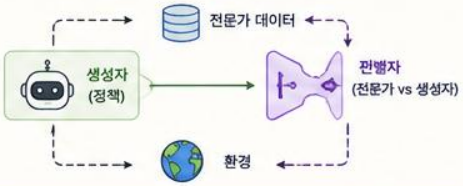
3) Inverse Reinforcement Learning (IRL)

전문가 행동을 가장 잘 설명하는 보상함수를 추정한 후, 그 보상으로부터 최적 정책을 학습



4) GAIL / AIRL

적대적 학습(Generative Adversarial)으로 전문가와 구분되지 않는 정책을 학습



5 장점

- ✓ 보상함수 설계가 필요 없음
- ✓ 학습이 비교적 간단하고 데이터 효율적(소량 데이터로도 가능)
- ✓ 실제 전문가의 지식/노하우를 직접 활용할 수 있음
- ✓ 안전한(검증된) 행동을 빠르게 습득 가능

6 한계

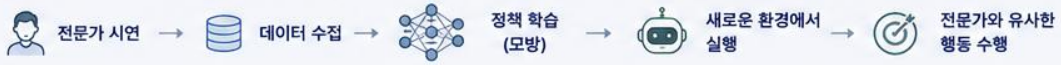
- ✗ 전문가가 하지 않은 상황에 일반화(Extrapolation) 어려움
- ✗ 전문가의 비최적 행동까지 그대로 따라할 수 있음
- ✗ 분포 차이(Distribution Shift)로 인해 성능 저하 가능
- ✗ 복잡한 장기적 목표 최적화 보장은 부족(BC의 한계)

7 활용 분야 (예시)



한눈에 정리

Imitation Learning은 "전문가의 행동을 관찰 → 모방 → 새로운 상황에서 유사하게 행동" 하는 학습 패러다임입니다.



참고: 강화학습과의 차이

- Imitation Learning: 보상 없이 전문가 데이터를 직접 모방
- Reinforcement Learning: 보상을 통해 시행착오를 거쳐 최적 정책 학습